

Structs und Speicherlayouts in C

In C kannst du mit einer *struct* mehrere Werte zu einem **zusammengehörtigen Datentyp** kombinieren. *structs* sind vergleichbar mit Objekten und Datensätzen in anderen Sprachen..

Beispiel: einen Struct erstellen

```
struct Punkt {  
    int x;  
    int y;  
};
```

Das ist ein neuer Typ mit zwei Ganzzahlen: *x* und *y*

Verwendung einer *struct*

```
struct Punkt p;  
p.x = 5;  
p.y = 10;  
  
printf("Punkt: (%d, %d)\n", p.x, p.y);
```

Du greifst it dem `.` Operator auf die Felder zu.

typedef – “saubere” Schreibweise

```
typedef struct {  
    int x;  
    int y;  
} Punkt;
```

Jetzt brauchst du nicht mehr den *struct* davor:

```
Punkt p1;
```

Ideal für eigene Libraries und API's – du kannst so einfach *ZFB_Pixel*, *ZFB_Buffer* usw. Deklarieren.

Verschachtelte *struct*

```
typedef struct {  
    int x, y;  
} Vektor2D;  
  
typedef struct {  
    Vektor2D position;  
    Vektor2D velocity;  
} Objekt;
```

Zugriff:

```
Objekt spieler;  
spieler.position.x = 100;  
spieler.velocity.y = 5;
```

Dynamische Allokation von *struct*

Du kannst Strukturen auch dynamisch auf dem Heap erzeugen:

```
Objekt* p = (Objekt*) malloc(sizeof(Objekt));  
if (p != NULL) {  
    p->position.x = 0; // statt (*p).position.x  
    p->velocity.y = 10;  
}
```

Der -> Operator wird verwendet bei **Zeigern auf Strukturen**.

Wie liegen *structs* im Speicher?

```
typedef struct {  
    char c;  
    int x;  
} Test;
```

- *char*: 1 Byte
- *int*: 4 Byte
- Aber `sizeof(Test)` ist oft **8 byte**, wegen **Padding**(zur Ausrichtung im Speicher)

Warum? CPUs laden oft Daten in 4-Byte_Blöcken. Damit *int* auf einer 4-Byte-Grenze beginnt, wird nach *char* 3 Byte Padding eingefügt.